

Boa noite Professora, tudo bem?

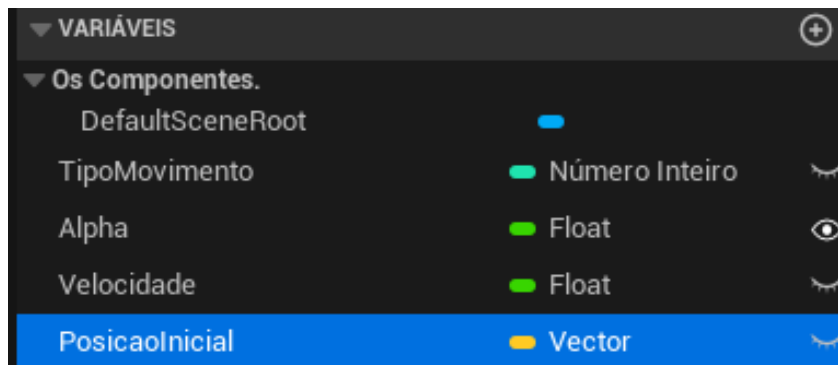
Decidi vincular esse descritivo do exercício junto, pois como a programação em Unreal é feita através de Blueprints, pode ser difícil de visualizar o “código” da melhor forma na engine.

O primeiro passo foi criar um cenário simples com itens que haviam na minha biblioteca fab. Usei uma temática praça com uma flor móvel, como se fosse o level de um game de plataforma 3D.

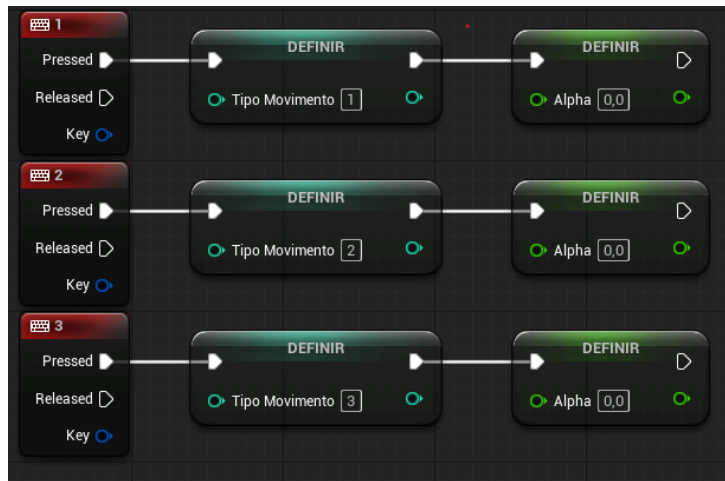


Optei por vincular a plataforma um blueprint com as 3 opções de movimentos, Linear, Bezier e Spline (esse último utilizando o recurso de movimentação incluso dentro da própria Unreal).

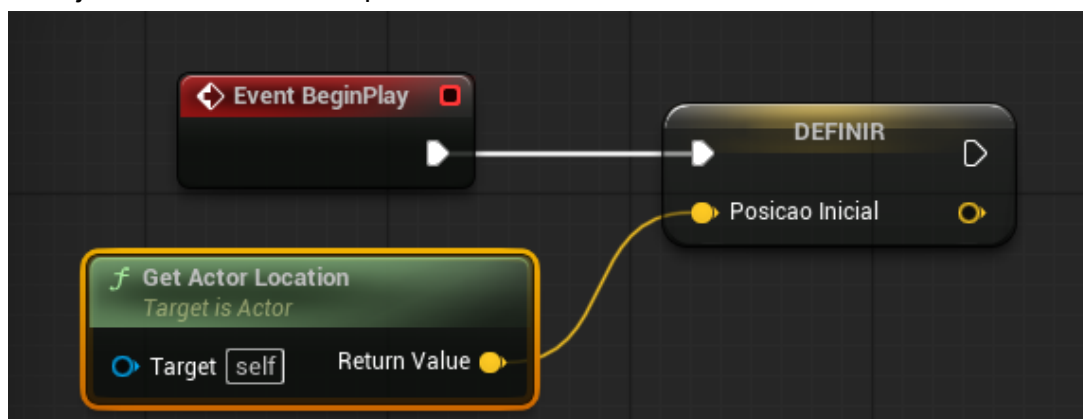
Já dentro do Blueprint, o caminho foi criar uma variável que escolhesse o tipo de movimento, e outras que realizassem a gestão do que iria ocorrer.



Já aqui é a definição do tipo de movimento que o objeto irá realizar de acordo com a tecla selecionada. E no final voltando a zero para que o movimento pare.

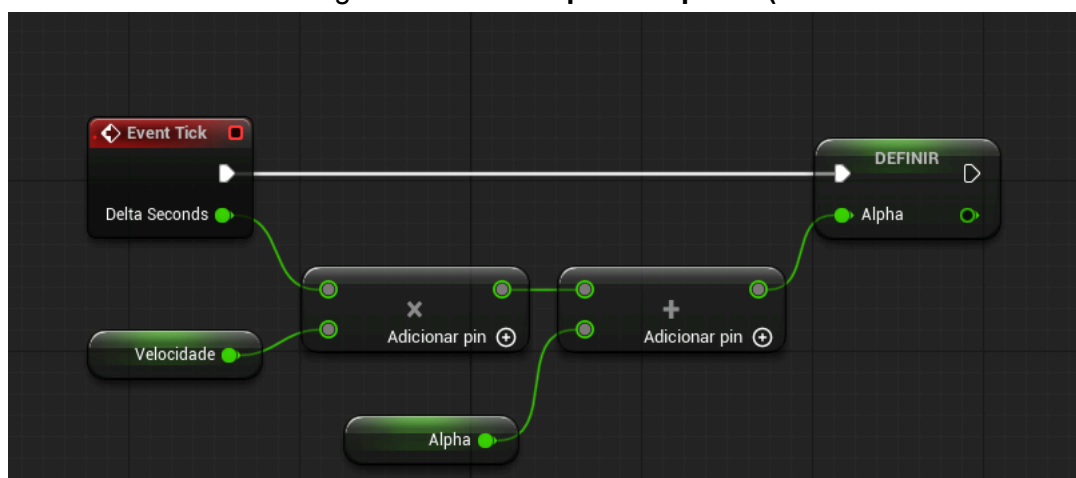


Já o comando abaixo é para ao iniciar do play do jogo, a posição inicial será a localização do objeto vinculado ao Blueprint no mundo

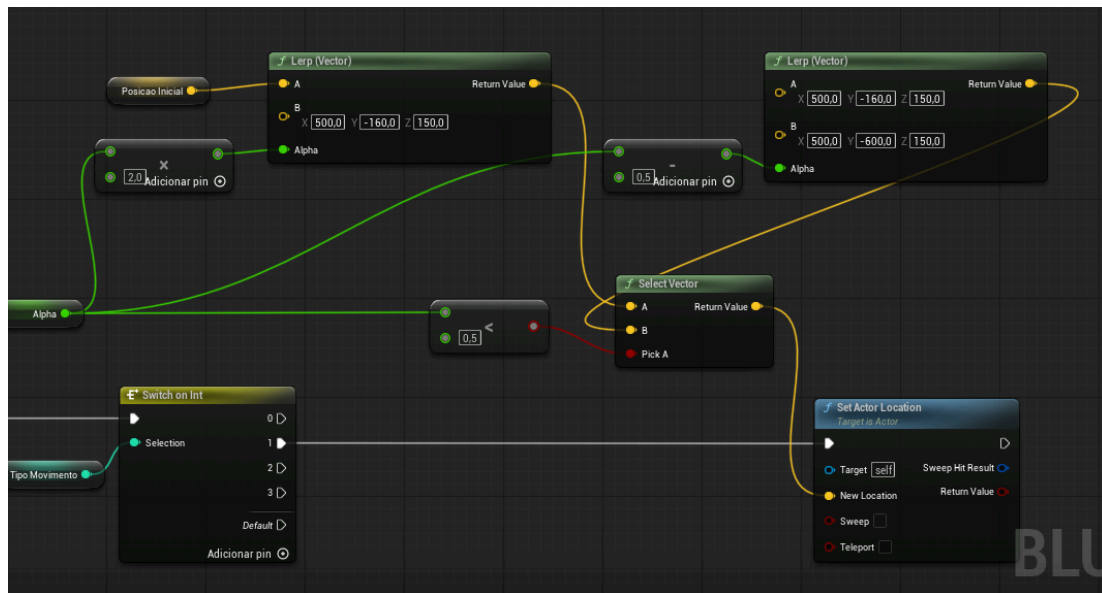


A função abaixo serve para que a velocidade seja linear com o delta time da máquina que estiver rodando o projeto, ficando assim um movimento uniforme.

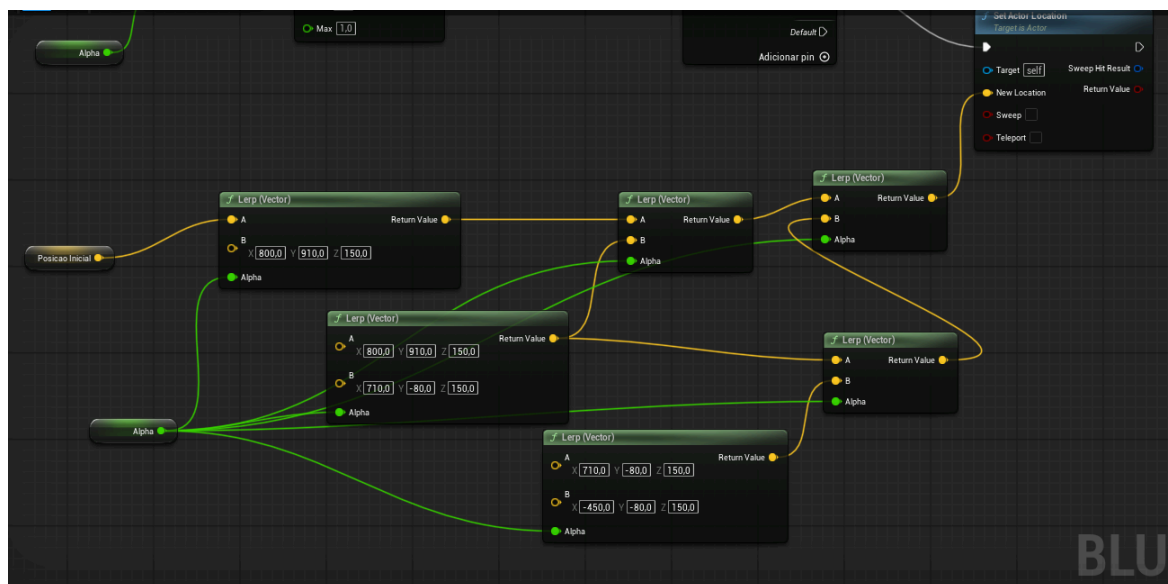
É a estrutura visual da seguinte fórmula: **Alpha = Alpha + (Delta Seconds * Velocidade)**



Para o case 1 a escolha foi a curva linear. Aqui tem dois movimentos, o primeiro em X, com o alpha abaixo de 0,5. Depois é movimentado em Y o objeto, e na finalização ele para, após o alpha passar de 1. Dessa maneira o objeto anda em uma espécie de “L” pelo cenário. Movimento bem simples, mas que já mostra uma animação rápida e dura da curva.

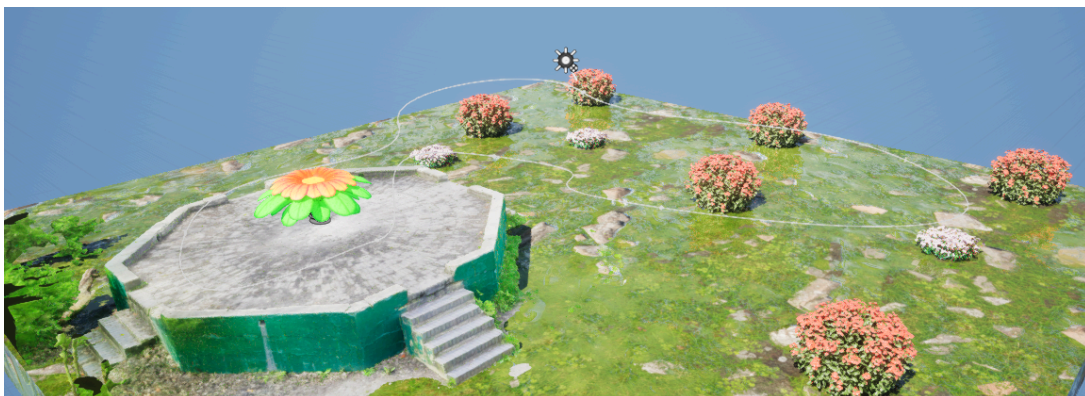
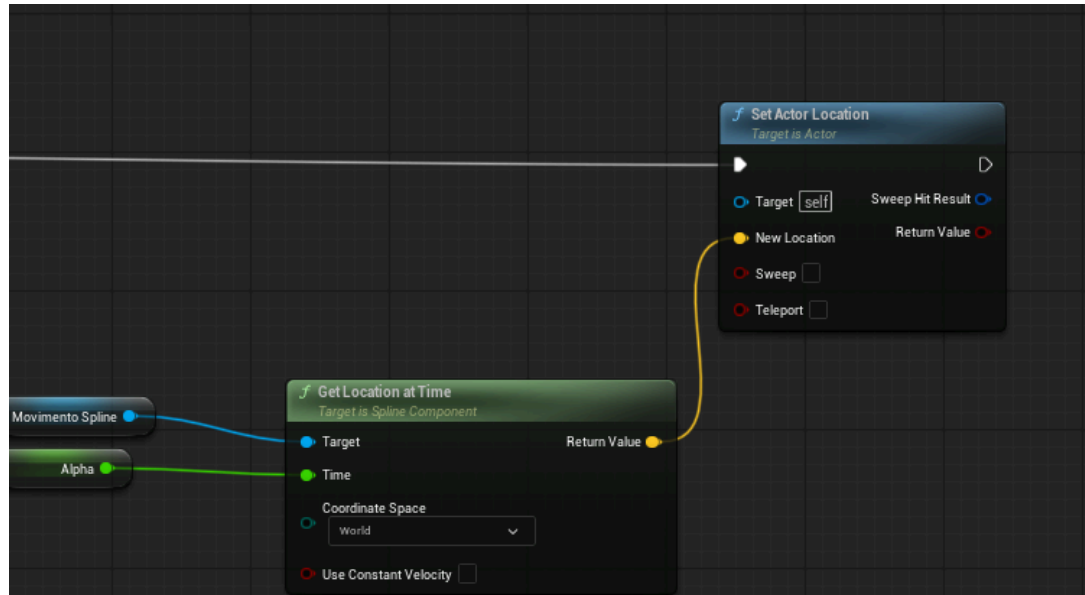


Pare o case 2, a curva de Bezier, o segredo foi criar 6 lerps (essas estruturas de vector inicio/fim). Conectei posição inicial com A, vector A com B, B com C, e depois C com com posição inicial. Essa estrutura “água viva”, permite com o objeto passeie pelos pontos de maneira leve, simulando a curva solicitada.



Para o case 3, testei usar o Spline para criar a trajetória do objeto. Ele até funciona, mas ao ir para a segunda volta o comando buga, por isso testei outra forma de edição, mas daí foquei somente no Spline.

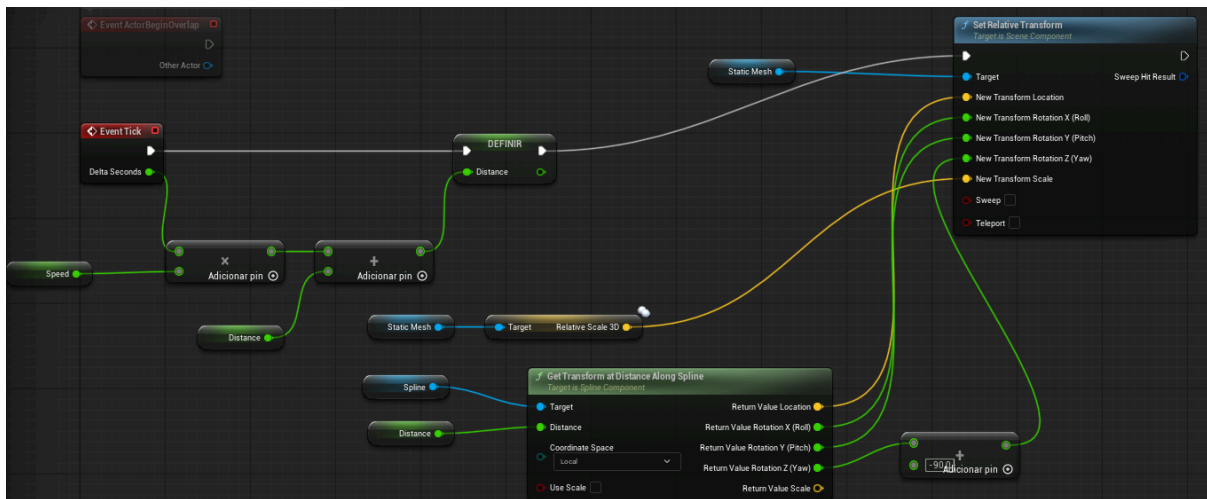
Esse é o modelo inicial com bug:



E esse é o modelo correto com uma “programação focada 100%no modelo Spline.

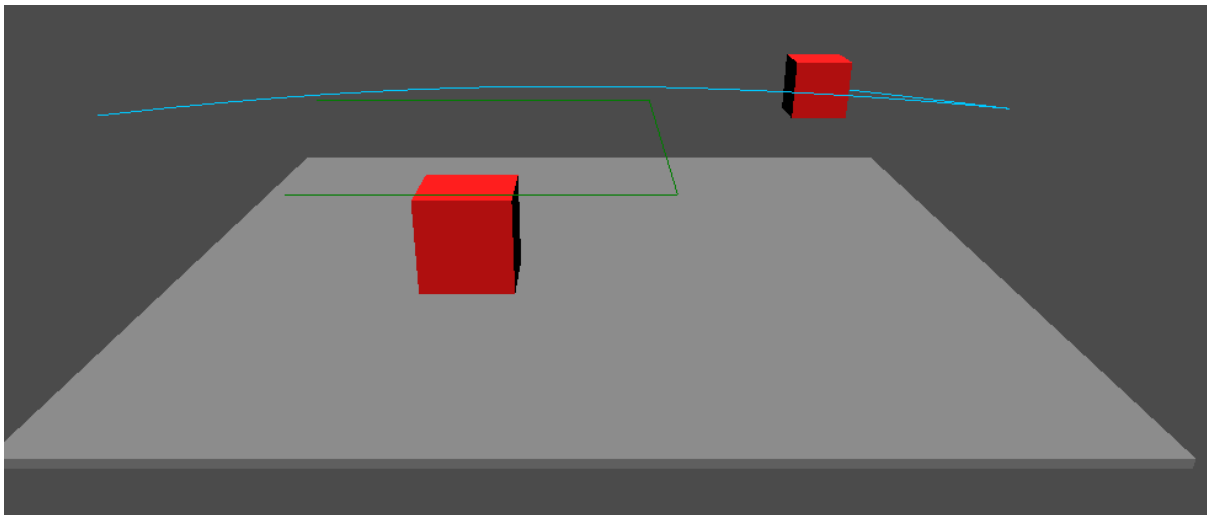


Nesse fluxograma já é setado velocidade e distância como 300, e rotação fixa de -90 no eixo Z. E depois desses parametros básicos, aí a Spline foi aplicada no tele principal com o desenho do percurso da imagem anterior.



Com esses 2 modelinhas de Blueprint, entrego as atividades solicitadas no trabalho 1.

Obs: como não programei “na mão”, mas sim por nodos, me senti na obrigação de realizar uma programação mais tradicional, pois acho que era a proposta do exercício. Então peguei o exemplo do cubo disponibilizado em aula, e criei para cada cubo um script para simular os movimentos solicitados. Toda a resolução desse passo a passo para construir o GDScript está no próprio código, mas também vou deixar o texto aqui caso queira ver mais detalhado.



CÓDIGO LINEAR

```
extends MeshInstance3D
```

```
# Called when the node enters the scene tree for the first time.
```

```
#func _ready() -> void:
```

```

#      pass # Replace with function body.
# Called every frame. 'delta' is the elapsed time since the previous frame.
#func _process(delta: float) -> void:
#      pass
#código acima comentado é o default de quando gera um novo script

#@onready var cubo: MeshInstance3D = $MeshIntance3D
#Referencia para o cubo que vou movimentar, porém só é utilizado se o script
#não estiver vinculado ao cubo. Precisei de algumas quebras de cabeça para
#descobrir isso.

#Quero que o cubo percorra um quadrado na tela
#então criei um vector com as posições a se percorrer
var pontos: Array[Vector3] = [
    Vector3(-2, 1, -2), #se o chão estiver em 0, tem que aumentar o Y
    Vector3(2, 1, -2), #se não o cubo se enterra e não da para ver
    Vector3(2, 1, 2), #as linhas sendo desenhadas no chão
    Vector3(-2, 1, 2)
]

var indice_origem: int = 0
#ponto da origem atual do array

var progresso: float = 0.0
#variavel de progresso na interpolação, vai de 0 a 1

@export var velocidade: float = 0.8
# Velocidade do cubo

var desenha_linha: ImmediateMesh
var material_linha: StandardMaterial3D
#variáveis para deenhar a linha na tela
#serve para acompanhar o movimento linear, para deixar bem visível

func _ready() -> void:
    desenha_linha = ImmediateMesh.new()
    #inicia a malha para desenhara linha

    #comandos abaixo são para criar o material e a cor da linha
    material_linha = StandardMaterial3D.new()
    material_linha.shading_mode = BaseMaterial3D.SHADING_MODE_UNSHADED
    material_linha.albedo_color = Color(0, 0.5, 0)

    #esse novo meshinstance é para poder desenhara na tela, sem esse trecho
    #não funciona
    var instancia_linha = MeshInstance3D.new()
    instancia_linha.mesh = desenha_linha

```

```
instancia_linha.material_override = material_linha
add_child(instancia_linha)
```

```
#agora é posicionar o cubo no primeiro ponto
global_position = pontos[0]
```

```
func _process(delta: float) -> void:
    progresso += delta * velocidade
    #processo muito parecido feito nos blueprints com os nodos

    #se o progresso atingir ou ultrapassar num1, vai pro seguimento seguinte
    if progresso >= 1.0:
        progresso = 0.0 #reseta a variável
        indice_origem = (indice_origem + 1) % pontos.size() #avança o índice

    var indice_destino: int = (indice_origem + 1) % pontos.size()
    #calcula o índice do próximo destino

    var pos_inicio: Vector3 = pontos[indice_origem]
    var pos_fim: Vector3 = pontos[indice_destino]
    #obtem os pontos de origem e destino para o percurso

    global_position = pos_inicio.lerp(pos_fim, progresso)
    #aqui to aplicando a interpolação linear (tbm parecido com os nodos da unreal)
    #necessário para calcular a nova posição do cubo

    #atualiza o desenho das linhas do quadrado
    #sem problemas chamar antes uma funcao que será montada após
    desenhar_trajetoria()

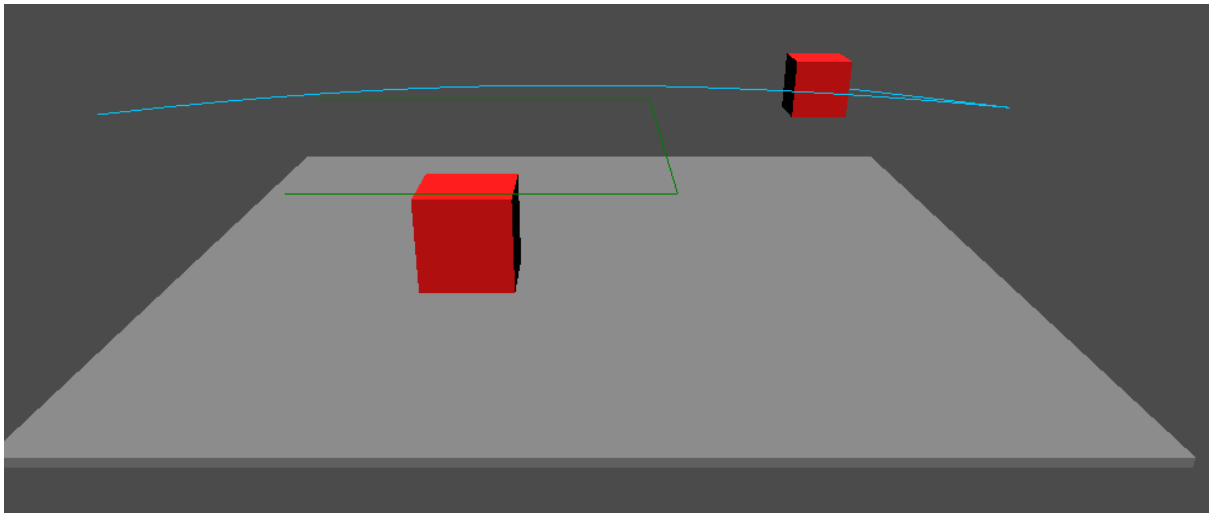
func desenhar_trajetoria() -> void:
    desenha_linha.clear_surfaces()
    #limpar os vértices anteriores que estiverem na malha da linha

    desenha_linha.surface_begin(Mesh.PRIMITIVE_LINES)
    #iniciar uma nova estrutura de linhas

    for i in range(indice_origem):
        desenha_linha.surface_add_vertex(pontos[i]) #Ponto inicial
        desenha_linha.surface_add_vertex(pontos[i + 1]) #Ponto final

    desenha_linha.surface_add_vertex(pontos[indice_origem]) #Inicio do segmento
    desenha_linha.surface_add_vertex(global_position) #Acompanhar cubo

    desenha_linha.surface_end()
```



CÓDIGO BEZIER

```
extends MeshInstance3D
```

```
#Montando os pontos da curva de BEZIER
```

```
var p0: Vector3 = Vector3(-8, 1.0, -4) # o ponto que vai iniciar a curva
```

```
var p1: Vector3 = Vector3(0, 1.0, -10) # Ponto que puxa a curva
```

```
var p2: Vector3 = Vector3(8, 1.0, -4) # Ponto final da curva
```

```
var progresso: float = 0.0
```

```
#variavel para controle
```

```
@export var velocidade: float = 0.5
```

```
#criar variáveis para desenhar a trajetória na tela
```

```
var desenha_linha: ImmediateMesh
```

```
var material_linha: StandardMaterial3D
```

```
#Num de segmentos para mandar a curva suave, usei 30 igual o exemplo que
```

```
#encontrei no youtube, gostei bastante da suavidade
```

```
var suavidade_desenho: int = 30
```

```
func _ready() -> void:
```

```
    #mesmo esquema do código linear para criação da linha
```

```
    desenha_linha = ImmediateMesh.new()
```

```
    material_linha = StandardMaterial3D.new()
```

```
    material_linha.shading_mode = BaseMaterial3D.SHADING_MODE_UNSHADED
```

```
    material_linha.albedo_color = Color(0, 0.8, 1) #usar Azul para diferenciar
```

```
    #criação da malha para o desenho da linha
```

```
    var instancia_linha = MeshInstance3D.new()
```

```
    instancia_linha.mesh = desenha_linha
```

```
    instancia_linha.material_override = material_linha
```



```

#deixar a linha independente do cubo
instancia_linha.top_level = true
add_child(instancia_linha)

#cubo agora vai para o ponto inicial da curva
global_position = p0

func _process(delta: float) -> void:
    #progresso é incrementado de 0 a 1, igual no código da linha linear
    progresso += delta * velocidade

    #reiniciar o ciclo quando chega n o fim da curva
    if progresso >= 1.0:
        progresso = 0.0 #não esquecer dos espaços corretos

    #CALCULO DA BEZIER -----!!!!!!
    global_position = p0.bezier_interpolate(p1, p1, p2, progresso) #utilizando a própria
funcao bezier da godot
    desenhar_trajetoria_bezier()
    #desenha o trajeto

func desenhar_trajetoria_bezier() -> void:
    desenha_linha.clear_surfaces()
    desenha_linha.surface_begin(Mesh.PRIMITIVE_LINES)

    #calcula dos passos da curva que já foram dados
    var passos_atuais: int = int(progresso * suavidade_desenho)
    var ponto_anterior: Vector3 = p0

    #for do desenho da curva
    for i in range(1, passos_atuais + 1):
        var t: float = float(i) / float(suavidade_desenho)
        var ponto_atual: Vector3 = p0.bezier_interpolate(p1, p2, p2, t) #calcula Bezier

        desenha_linha.surface_add_vertex(ponto_anterior)
        desenha_linha.surface_add_vertex(ponto_atual)

        ponto_anterior = ponto_atual
        #código para "surfar" de um ponto para o próximo

    #passa os pontos para o desenha linha
    desenha_linha.surface_add_vertex(ponto_anterior)
    desenha_linha.surface_add_vertex(global_position)

    desenha_linha.surface_end()

```